

TRABALHO 6 EDA1

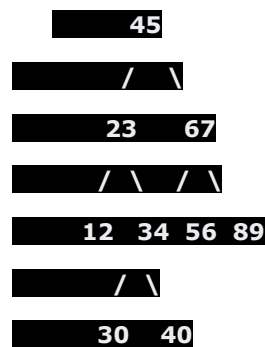
Árvores Binárias de Pesquisa

ALUNO: RODRIGO DUTRA FERREIRA;
MATRÍCULA: 242015951;

Questão 1: Construção e Propriedades

Sequência de inserção: 45, 23, 67, 12, 34, 56, 89, 30, 40

Árvore resultante:



Explicação da construção: 45 é inserido como raiz. 23 é menor que 45, vai para a esquerda de 45. 67 é maior que 45, vai para a direita de 45. 12 é menor que 45 e menor que 23, vai para a esquerda de 23. 34 é menor que 45 e maior que 23, vai para a direita de 23. 56 é maior que 45 e menor que 67, vai para a esquerda de 67. 89 é maior que 45 e maior que 67, vai para a direita de 67. 30 é menor que 45, maior que 23, menor que 34, vai para a esquerda de 34. 40 é menor que 45, maior que 23, maior que 34, vai para a direita de 34.

Raiz: 45.

Nós folha (sem filhos): 12, 30, 40, 56, 89.

Nós internos (com pelo menos um filho): 45, 23, 67, 34.

Altura da árvore: considerando a raiz no nível 0, o caminho mais longo é 45, 23, 34, 30 (ou 40), o que dá altura igual a 3.

Questão 2: Análise de Percursos

Pré-ordem (raiz, esquerda, direita): 45, 23, 12, 34, 30, 40, 67, 56, 89

Em-ordem (esquerda, raiz, direita): 12, 23, 30, 34, 40, 45, 56, 67, 89

Pós-ordem (esquerda, direita, raiz): 12, 30, 40, 34, 23, 56, 89, 67, 45

O percurso em-ordem é particularmente útil em Árvores Binárias de Pesquisa porque, dada a propriedade de ordenação da ABP, onde toda subárvore esquerda contém valores menores que a raiz e toda subárvore direita contém valores maiores, percorrer os nós nessa ordem sempre produz a sequência de chaves em ordem crescente, sem necessidade de qualquer etapa adicional de ordenação. Isso torna esse percurso a forma natural de listar os elementos ordenados ou de verificar se a árvore realmente respeita a propriedade de uma ABP.

Questão 3: Implementação em C

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

//questao 3

typedef struct TNoBin {

    int chave;

    struct TNoBin *esquerda;

    struct TNoBin *direita;

} TNoBin;

TNoBin *criarNo(int chave) {

    TNoBin *novo = (TNoBin *) malloc(sizeof(TNoBin));

    novo->chave = chave;

    novo->esquerda = NULL;

    novo->direita = NULL;

    return novo;
```

```

}

TNoBin *inserir(TNoBin *raiz, int chave) {

    if (raiz == NULL) {

        return criarNo(chave);

    }

    if (chave < raiz->chave) {

        raiz->esquerda = inserir(raiz->esquerda, chave);

    } else if (chave > raiz->chave) {

        raiz->direita = inserir(raiz->direita, chave);

    }

    return raiz;

}

```

```

TNoBin *buscar(TNoBin *raiz, int chave) {

    if (raiz == NULL || raiz->chave == chave) {

        return raiz;

    }

    if (chave < raiz->chave) {

        return buscar(raiz->esquerda, chave);

    }

    return buscar(raiz->direita, chave);

}

```

```
void preOrdem(TNoBin *raiz) {  
  
    if (raiz == NULL) return;  
  
    printf("%d ", raiz->chave);  
  
    preOrdem(raiz->esquerda);  
  
    preOrdem(raiz->direita);  
  
}
```

```
void emOrdem(TNoBin *raiz) {  
  
    if (raiz == NULL) return;  
  
    emOrdem(raiz->esquerda);  
  
    printf("%d ", raiz->chave);  
  
    emOrdem(raiz->direita);  
  
}
```

```
void posOrdem(TNoBin *raiz) {  
  
    if (raiz == NULL) return;  
  
    posOrdem(raiz->esquerda);  
  
    posOrdem(raiz->direita);  
  
    printf("%d ", raiz->chave);  
  
}
```

```
int altura(TNoBin *raiz) {
```

```

    if (raiz == NULL) return -1;

    int he = altura(raiz->esquerda);

    int hd = altura(raiz->direita);

    return 1 + (he > hd ? he : hd);
}

void liberarArvore(TNoBin *raiz) {

    if (raiz == NULL) return;

    liberarArvore(raiz->esquerda);

    liberarArvore(raiz->direita);

    free(raiz);

}

// Questao 4

typedef struct TNoInventario {

    int codigoPatrimonio;

    char nome[50];

    struct TNoInventario *esquerda;

    struct TNoInventario *direita;

} TNoInventario;

TNoInventario *criarNoInventario(int codigo, const char *nome) {

```

```

TNoInventario *novo = (TNoInventario *) malloc(sizeof(TNoInventario));

novo->codigoPatrimonio = codigo;

strncpy(novo->nome, nome, sizeof(novo->nome) - 1);

novo->nome[sizeof(novo->nome) - 1] = '\0';

novo->esquerda = NULL;

novo->direita = NULL;

return novo;
}

TNoInventario *inserirInventario(TNoInventario *raiz, int codigo, const char *nome) {

    if (raiz == NULL) {

        return criarNoInventario(codigo, nome);

    }

    if (codigo < raiz->codigoPatrimonio) {

        raiz->esquerda = inserirInventario(raiz->esquerda, codigo, nome);

    } else if (codigo > raiz->codigoPatrimonio) {

        raiz->direita = inserirInventario(raiz->direita, codigo, nome);

    }

    return raiz;
}

/* Recebe a raiz e um codigo de patrimonio, retorna o nome do equipamento

ou "Nao encontrado" caso o codigo nao exista na arvore. */

```

```

const char *buscarEquipamento(TNoInventario *raiz, int codigo) {

    if (raiz == NULL) {

        return "Nao encontrado";

    }

    if (codigo == raiz->codigoPatrimonio) {

        return raiz->nome;

    }

    if (codigo < raiz->codigoPatrimonio) {

        return buscarEquipamento(raiz->esquerda, codigo);

    }

    return buscarEquipamento(raiz->direita, codigo);

}

void liberarInventario(TNoInventario *raiz) {

    if (raiz == NULL) return;

    liberarInventario(raiz->esquerda);

    liberarInventario(raiz->direita);

    free(raiz);

}

//Testes

int main(void) {

```

```
/* Testes da Questao 3, usando a sequencia da Questao 1 */
```

```
TNoBin *raiz = NULL;
```

```
int chaves[] = {45, 23, 67, 12, 34, 56, 89, 30, 40};
```

```
int n = sizeof(chaves) / sizeof(chaves[0]);
```

```
for (int i = 0; i < n; i++) {
```

```
    raiz = inserir(raiz, chaves[i]);
```

```
}
```

```
printf("Pre-ordem: ");
```

```
preOrdem(raiz);
```

```
printf("\n");
```

```
printf("Em-ordem: ");
```

```
emOrdem(raiz);
```

```
printf("\n");
```

```
printf("Pos-ordem: ");
```

```
posOrdem(raiz);
```

```
printf("\n");
```

```
printf("Altura da arvore: %d\n", altura(raiz));
```



```

TNoBin *resultado = buscar(raiz, 34);

if (resultado != NULL) {

    printf("Busca por 34: encontrado (chave %d)\n", resultado->chave);

} else {

    printf("Busca por 34: nao encontrado\n");

}

resultado = buscar(raiz, 100);

if (resultado != NULL) {

    printf("Busca por 100: encontrado (chave %d)\n", resultado->chave);

} else {

    printf("Busca por 100: nao encontrado\n");

}

liberarArvore(raiz);

raiz = NULL;

printf("\n");

/* Testes da Questao 4 */

TNoInventario *inventario = NULL;

inventario = inserirInventario(inventario, 1050, "Monitor Dell 24pol");

inventario = inserirInventario(inventario, 1020, "Teclado Logitech");

```

```
    inventario = inserirInventario(inventario, 1080, "Notebook Lenovo");

    inventario = inserirInventario(inventario, 1010, "Mouse sem fio");

    inventario = inserirInventario(inventario, 1035, "Estabilizador");


    printf("Busca codigo 1035: %s\n", buscarEquipamento(inventario, 1035));

    printf("Busca codigo 1080: %s\n", buscarEquipamento(inventario, 1080));

    printf("Busca codigo 9999: %s\n", buscarEquipamento(inventario, 9999));


    liberarInventario(inventario);

    inventario = NULL;


    return 0;

}
```

Saída do Terminal:

Pre-ordem: 45 23 12 34 30 40 67 56 89

Em-ordem: 12 23 30 34 40 45 56 67 89

Pos-ordem: 12 30 40 34 23 56 89 67 45

Altura da arvore: 3

Busca por 34: encontrado (chave 34)

Busca por 100: nao encontrado

Busca codigo 1035: Estabilizador

Busca código 1080: Notebook Lenovo

Busca código 9999: Não encontrado

Questão 4: Caso Prático — Sistema de Inventário de Laboratório

A estrutura do nó, definida no código como `TNoInventario`, guarda o código de patrimônio como inteiro (que serve de chave de ordenação da árvore), o nome do equipamento como uma cadeia de caracteres, e os ponteiros para os filhos esquerdo e direito, seguindo o mesmo princípio de uma ABP comum. A inserção segue a lógica padrão de comparação pelo código de patrimônio. A função `buscarEquipamento` recebe a raiz da árvore e o código de patrimônio desejado, percorre a árvore recursivamente comparando o código buscado com o código do nó atual, indo para a esquerda se for menor e para a direita se for maior, e retorna o nome do equipamento quando encontra o nó correspondente, ou a mensagem "Não encontrado" caso chegue a um ponteiro nulo sem localizar o código. O código completo dessa função está no arquivo `main.c`, junto com a implementação da Questão 3.

Questão 5: Análise de Balanceamento

Sequência A (10, 20, 30, 40, 50): como cada novo valor inserido é sempre maior que o anterior, cada elemento se torna filho direito do elemento anterior. O resultado é uma árvore completamente degenerada, na prática equivalente a uma lista encadeada, com altura igual a 4 ($n-1$ para $n=5$ elementos). Isso significa que, no pior caso, uma busca precisa percorrer todos os n nós da árvore, tornando a complexidade de tempo da busca $O(n)$.

Sequência B (30, 10, 40, 20, 50): 30 é a raiz; 10 é menor, vai para a esquerda de 30; 40 é maior, vai para a direita de 30; 20 é menor que 30 e maior que 10, vai para a direita de 10; 50 é maior que 30 e maior que 40, vai para a direita de 40. O resultado é uma árvore bem mais equilibrada, com altura igual a 2, já que os valores foram inseridos em uma ordem que distribui os elementos entre as duas subárvores da raiz. Nesse caso, a busca por qualquer elemento percorre no máximo 3 nós, o que aproxima a complexidade de tempo de $O(\log n)$.

A diferença estrutural entre as duas árvores mostra que o desempenho de uma ABP depende diretamente da ordem de inserção dos elementos, e não apenas do conjunto de valores armazenados. Quando os dados chegam já ordenados (como na Sequência A), a árvore degenera e perde a vantagem logarítmica que é a principal razão de se usar uma ABP. Quando a ordem de inserção favorece uma distribuição mais equilibrada entre subárvores esquerda e direita (como na Sequência B), a altura da árvore fica próxima de $\log_2(n)$, garantindo que as operações de busca, inserção e remoção sejam eficientes.